ZAKPY ACADEMY
Votre réussite, notre mission.

SUPPORT DE COURS

TP1 : Mesure de température
LM35 et afficheur 7 segments

Arduino Mega 2560, ADC, multiplexage, timers et interruptions

CARTE	OUTILS	QUESTIONS TRAITÉES
Arduino Mega 2560 ATmega2560 à 16 MHz	**Arduino IDE et Proteus** ou Wokwi sur le navigateur	**2.1 à 2.5** du sujet du TP

0 Présentation du TP

Dans ce TP on mesure la température avec un capteur LM35, et on l'affiche sur 3 afficheurs 7 segments avec une précision de 0,1 °C (format xx.x). Un potentiomètre permet de régler une température d'alarme entre 0 et 40 °C. Quand la température dépasse cette valeur, une LED se met à clignoter.

Voici le chemin suivi par la mesure, puis par le seuil d'alarme :

LM35 10 mV par °C	**Entrée A0** ADC : N de 0 à 1023	**Calcul** N × 1100 / 1024	**prepareAffichage** 3 codes	**Timer 1** affichage, 5 ms
Potentiomètre 0 à 1,1 V	**Entrée A1** ADC : N de 0 à 1023	**map()** seuil de 0 à 400	**temp > seuil** alarme	**Timer 3** LED, 0,5 s

ÉLÉMENT	BROCHE ARDUINO	REMARQUE
LM35 (sortie Vout)	`A0`	alimenté en 5 V
Potentiomètre (curseur)	`A1`	alimenté en 1,1 V dans Proteus
Segments A, B, C, D, E, F, G, DP	`22 à 29`	c'est le PORTA, bits 0 à 7
Afficheurs dizaines, unités, dixièmes	`52, 51, 50`	l'afficheur de gauche n'est pas câblé
LED d'alarme	`32`	HIGH = allumée

REMARQUE

Dans tout le programme, `loop()` s'occupe de mesurer et de décider. Le reste est fait par les timers : le Timer 1 rafraîchit l'affichage toutes les 5 ms et le Timer 3 fait clignoter la LED toutes les 0,5 s.

1 Mesure de la température

Q2.1 Le capteur LM35

Le LM35 a 3 broches : +5 V, Vout et GND. Sa sortie donne 10 mV pour chaque degré, à partir de 0 °C. Il n'y a pas de calibration à faire.

TEMPÉRATURE	0 °C	24,8 °C	37 °C	100 °C
Vout	0 mV	248 mV	370 mV	1000 mV

ATTENTION

Il ne faut pas confondre deux tensions. Le LM35 est alimenté en 5 V, c'est ce qui le fait fonctionner. La référence de l'ADC est de 1,1 V, c'est l'échelle avec laquelle l'Arduino mesure la tension. Elle est interne au microcontrôleur.

Le convertisseur analogique numérique (ADC)

L'Arduino ne lit pas directement des volts, il lit un nombre. Son ADC est sur 10 bits, donc il donne un nombre N entre 0 et 1023 (2^{10} = 1024 valeurs). La tension se retrouve avec :

$$V = N \times V_{ref} / 1024$$

Pourquoi on choisit la référence 1,1 V

La référence découpe l'intervalle en 1024 pas. Plus elle est petite, plus chaque pas est fin. Comme le LM35 ne dépasse pas 1 V, on prend la plus petite :

RÉFÉRENCE	1 PAS	EN TEMPÉRATURE	PRÉCISION DE 0,1 °C ?
5 V (DEFAULT)	5000 / 1024 = 4,88 mV	0,49 °C	non
2,56 V (INTERNAL2V56)	2560 / 1024 = 2,5 mV	0,25 °C	non
1,1 V (INTERNAL1V1)	**1100 / 1024 = 1,07 mV**	**0,107 °C**	**oui**

Le calcul se fait en deux règles de trois. L'ADC donne 1 pas = 1,07 mV. Ensuite le LM35 donne 10 mV pour 1 °C, donc 1,07 mV correspond à environ 0,1 °C. Avec cette référence on peut mesurer jusqu'à 1100 / 10 = 110 °C, ce qui est largement suffisant.

Le calcul dans le programme

Le calcul N × 1100 / 1024 donne la tension en millivolts. Et comme 1 mV correspond à 0,1 °C, ce nombre est directement la température en dixièmes de degré. Par exemple pour 24,8 °C, le LM35 sort 248 mV, l'ADC donne N ≈ 231, et 231 × 1100 / 1024 = 248.

```
int mesureTemp() {
  long N = analogRead(LM35_pin);          // N entre 0 et 1023
  return (int)((N * 1100L) / 1024L);      // 248 = 24,8 °C
}
```

ATTENTION AU TYPE INT

Sur Arduino, un `int` ne dépasse pas 32 767. Or 231 × 1100 = 254 100. Si on oublie le `long` et le `L`, le résultat est faux, et le compilateur ne donne aucune erreur.

Pourquoi on garde 248 au lieu de 24,8

Une variable `int` ne garde que des entiers. On compte donc en dixièmes de degré, un peu comme on compte en centimes au lieu de dirhams. Pour afficher la valeur, on la découpe avec la division entière et le modulo (le reste de la division) :

OPÉRATION	POUR 248	CE QU'ON OBTIENT
`t / 10`	24	la partie entière
`t % 10`	8	les dixièmes
`t / 100, (t / 10) % 10, t % 10`	2, 4, 8	les 3 chiffres, un pour chaque afficheur

ASTUCE

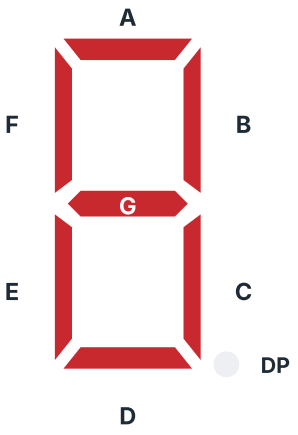
Juste après `analogReference()`, la première lecture peut être fausse, le temps que la référence se stabilise. On fait donc une première lecture qu'on n'utilise pas dans `setup()`.

2 L'afficheur 7 segments

Un chiffre, c'est 8 LED

Chaque segment est une LED, et chacune a sa lettre. On commence par A en haut, puis on tourne dans le sens des aiguilles d'une montre jusqu'à F. G est le segment du milieu et DP est le point.

Pour afficher un chiffre, il suffit de choisir les segments à allumer. Par exemple le 2, c'est A, B, D, E et G.



Nom des segments

Coder un chiffre sur un octet

On a 8 segments et un octet contient 8 bits, donc on donne un bit à chaque segment. Un bit à 1 veut dire que le segment est allumé (du point de vue de l'Arduino).

BIT	7	6	5	4	3	2	1	0
Segment	DP	G	F	E	D	C	B	A
Broche	29	28	27	26	25	24	23	22
Le 2 = 0x5B	**0**	**1**	**0**	**1**	**1**	**0**	**1**	**1**

CHIFFRE	SEGMENTS	CODE (1 = ALLUMÉ)	CODE ANODE COMMUNE (PROF)
0	A B C D E F	`0x3F`	`0xC0`
1	B C	`0x06`	`0xF9`
2	A B D E G	`0x5B`	`0xA4`
3	A B C D G	`0x4F`	`0xB0`
4	B C F G	`0x66`	`0x99`
5	A C D F G	`0x6D`	`0x92`
6	A C D E F G	`0x7D`	`0x82`
7	A B C	`0x07`	`0xF8`
8	A B C D E F G	`0x7F`	`0x80`
9	A B C D F G	`0x6F`	`0x90`

Pour ajouter le point, on met le bit 7 à 1 avec `| 0x80`. Par exemple « 4. » donne `0x66 | 0x80 = 0xE6`.

Le PORTA

Les broches 22 à 29 correspondent exactement au PORTA du microcontrôleur, dans l'ordre de A jusqu'à DP. Donc on peut écrire les 8 segments en une seule ligne. `DDRA` et `PORTA` ne sont pas des variables qu'on déclare, ce sont des registres du microcontrôleur, reliés directement aux broches.

REGISTRE	RÔLE	BIT À 1	ÉQUIVALENT ARDUINO
DDRA	sens des broches	sortie	`pinMode(…, OUTPUT)`
PORTA	valeur envoyée	5 V	`digitalWrite(…, HIGH)`
PINA	lecture des broches	broche à 5 V	`digitalRead(…)`

```
DDRA  = 0xFF;   // les 8 broches 22..29 en sortie
PORTA = 0x5B;   // affiche un 2
```

Anode commune, cathode commune et le 4049

Une LED s'allume quand son anode est au + et sa cathode au −. Dans un afficheur, les 8 LED d'un même chiffre partagent une broche, qu'on appelle la broche commune.

TYPE	LA BROCHE COMMUNE (DIGIT)	LES SEGMENTS	UN SEGMENT S'ALLUME SI…
Anode commune (notre TP)	c'est l'anode (+)	ce sont les cathodes (−)	sa broche est à 0
Cathode commune	c'est la cathode (−)	ce sont les anodes (+)	sa broche est à 1

Dans Proteus, il y a des inverseurs 4049 entre l'Arduino et l'afficheur, sur les segments et aussi sur les digits. Chaque inverseur transforme 1 en 0 et 0 en 1. Du côté de l'Arduino, ça donne :

> **À RETENIR**
> Un segment à HIGH est allumé, et un digit à LOW est sélectionné.
> L'afficheur est à anode commune, mais avec les 4049 il se commande comme une cathode commune. C'est pour ça qu'on utilise la table `0x3F, 0x06…` . On peut aussi prendre la table du prof et l'inverser avec `~` .

SEGMENT (ARDUINO)	DIGIT (ARDUINO)	RÉSULTAT
HIGH	**LOW**	**le segment s'allume**
HIGH	HIGH	éteint, l'afficheur n'est pas sélectionné
LOW	LOW	éteint, le segment n'est pas demandé
LOW	HIGH	éteint

En résumé, le segment choisit les barres à allumer et le digit choisit l'afficheur. La LED ne s'allume que si les deux sont bons en même temps.

3 Le multiplexage

Q2.2 Le problème

Le module a 12 broches pour 4 afficheurs : 8 broches de segments partagées par tous les afficheurs, et une broche par afficheur. Si on sélectionne les 3 afficheurs en même temps, ils affichent tous le même chiffre.

La solution

On allume un seul afficheur avec son chiffre, ensuite le suivant, puis le suivant, et on recommence sans arrêt. Si on le fait plus de 50 fois par seconde environ, l'œil ne voit plus le clignotement et on a l'impression que les 3 chiffres sont allumés en même temps. C'est la persistance rétinienne.

TEMPS	PORTA	BROCHE 52	BROCHE 51	BROCHE 50	CE QU'ON VOIT
0 ms	`0x5B`	**0**	1	1	`2 _ _`
5 ms	`0xE6`	1	**0**	1	`_ 4. _`
10 ms	`0x4F`	1	1	**0**	`_ _ 3`
15 ms	on recommence. Un cycle dure 3 × 5 = 15 ms, soit environ 67 fois par seconde, et on voit 24.3				

L'ORDRE EST IMPORTANT

À chaque étape, on éteint d'abord tous les afficheurs, ensuite on écrit le code sur PORTA, et enfin on allume l'afficheur suivant.
Si on change PORTA alors que l'afficheur précédent est encore allumé, il montre un instant le mauvais chiffre. On appelle ça l'effet fantôme.

REMARQUE

Le multiplexage doit être répété en permanence. Si on le fait avec une boucle et des `delay()`, le programme ne peut rien faire d'autre, et dès que la boucle s'arrête l'affichage s'éteint. C'est pour ça que le TP utilise le Timer 1.

Les variables de l'affichage

VARIABLE	À QUOI ELLE SERT	POUR 24.8
`pinDigits[3]`	donne un numéro (0, 1, 2) à chaque broche d'afficheur	`{52, 51, 50}`
`affichage[3]`	garde les 3 codes à afficher. `loop()` les remplit et le timer les lit	`{0x5B, 0xE6, 0x7F}`
`digitCourant`	se souvient de l'afficheur à allumer au prochain passage	`0, 1, 2, 0…`

`const` veut dire que la valeur ne change jamais (le câblage ne bouge pas). `byte` est un nombre de 0 à 255 sur un octet. `volatile` est obligatoire quand une variable est utilisée dans une interruption, pour que le compilateur la relise toujours en mémoire.

Préparer les 3 codes à partir de la température

```
void prepareAffichage(int number) {        // number = 248 pour 24,8 °C
  if (number > 999) number = 999;          // 3 chiffres, donc 99,9 au maximum
  if (number < 0)   number = 0;

  int dizaine = number / 100;              // 2
  int unite   = (number / 10) % 10;        // 4
  int dixieme = number % 10;               // 8

  if (dizaine == 0) affichage[0] = 0x00;   // affiche " 9.5" et pas "09.5"
  else              affichage[0] = tabDigit[dizaine];
  affichage[1] = tabDigit[unite] | 0x80;   // le 4 avec le point
  affichage[2] = tabDigit[dixieme];
}
```

DEUX ERREURS QU'ON FAIT SOUVENT

Écrire `0x08` au lieu de `0x80` : ça allume le segment D (bit 3) au lieu du point (bit 7).
Écrire `affichage[0] = 2;` : on envoie le chiffre au lieu de son code, et seul le segment B s'allume. Il faut passer par `tabDigit[2]`.

4 Les timers et les interruptions

C'est quoi un timer

Un timer est un compteur qui se trouve dans le microcontrôleur. Il augmente tout seul au rythme de l'horloge, sans ralentir le programme. Si on devait l'écrire en code, ça ressemblerait à ça :

```
// Ce que le matériel fait tout seul (on ne l'écrit jamais)
// à chaque tick :
TCNT1 = TCNT1 + 1;             // le compteur augmente
if (TCNT1 > OCR1A) {           // la valeur cible est atteinte
  TCNT1 = 0;                   // retour à 0 (mode CTC)
  ISR(TIMER1_COMPA_vect);      // la fonction d'interruption est appelée
}
```

Une interruption, c'est quand le processeur met en pause ce qu'il fait (même pendant un `delay()`), exécute la fonction ISR, puis reprend là où il s'était arrêté. On n'appelle jamais une ISR nous-mêmes, c'est le matériel qui l'appelle.

Les 6 timers de l'ATmega2560

TIMER	TAILLE	UTILISATION PAR ARDUINO
Timer 0	8 bits	`millis()` et `delay()`, il ne faut pas le modifier
Timer 1	16 bits	PWM sur 11, 12, 13. Dans le TP on l'utilise pour l'affichage
Timer 2	8 bits	PWM sur 9, 10 et `tone()`
Timer 3	16 bits	PWM sur 2, 3, 5. Dans le TP on l'utilise pour la LED
Timer 4 et 5	16 bits	PWM sur 6, 7, 8 et 44, 45, 46 (le Timer 5 sert aussi à la librairie Servo)

Les timers 1, 3, 4 et 5 sont identiques : mêmes registres et mêmes bits, seul le numéro change. Chaque timer n'a qu'une seule période, c'est pour ça qu'on en utilise deux (5 ms pour l'affichage et 0,5 s pour la LED).

Comment calculer le prescaler et la valeur cible

Le temps est égal au nombre de ticks multiplié par la durée d'un tick. Le prescaler P divise l'horloge de 16 MHz, donc il fixe la durée d'un tick. La valeur cible OCR fixe le nombre de ticks.

PRESCALER P	TICKS PAR SECONDE (16 MHZ / P)	DURÉE D'UN TICK (P / 16 MS)	TEMPS MAX (65 536 TICKS)
1	16 000 000	0,0625 µs	4,1 ms
8	2 000 000	0,5 µs	32,8 ms
64	**250 000**	**4 µs**	**262 ms**
256	**62 500**	**16 µs**	**1,05 s**
1024	15 625	64 µs	4,19 s

1. On choisit un prescaler dont le temps max est supérieur au temps voulu. Le plus petit possible donne la meilleure précision.
2. On calcule le nombre de ticks : N = (16 000 000 / P) × temps. Il doit rester inférieur à 65 535, et c'est mieux s'il est entier.
3. On met OCR = N − 1, parce que le compteur commence à 0.
4. On vérifie : temps = (OCR + 1) × P / 16 000 000.

BESOIN	PRESCALER	TICKS	VALEUR CIBLE	BITS CS
Timer 1, 5 ms	64 (8 marche aussi)	250 000 × 0,005 = 1 250	`OCR1A = 1249`	`CS11 | CS10`
Timer 3, 0,5 s	256 (avec 64 il faudrait 125 000 ticks)	62 500 × 0,5 = 31 250	`OCR3A = 31249`	`CS32`

Les opérations sur les bits

ÉCRITURE	CE QU'ELLE FAIT	EXEMPLE AVEC X = 0000 1011
`1 << n`	met un 1 à la position n	`1 << 3 = 0000 1000`
`x |= masque`	met ces bits à 1 sans toucher aux autres	`x |= (1 << 2) donne 0000 1111`
`x &= ~masque`	met ces bits à 0 sans toucher aux autres	`x &= ~(1 << 3) donne 0000 0011`

`x |= m` est la même chose que `x = x | m`. On garde donc l'ancienne valeur de x. Avec un simple `=`, on effacerait les 8 bits.

Les registres du Timer 1

NOM	SIGNIFICATION	RÔLE
`TCCR1A, TCCR1B`	Timer/Counter Control Register 1, A et B	les réglages du timer
`TCNT1`	Timer/Counter 1	le compteur
`OCR1A`	Output Compare Register 1 A	la valeur cible
`TIMSK1`	Timer Interrupt Mask 1	les interruptions autorisées
`WGM12`	Waveform Generation Mode, bit 2	le mode CTC (retour à 0 à la cible)
`CS12, CS11, CS10`	Clock Select	le prescaler, et le démarrage du timer
`OCIE1A`	Output Compare Interrupt Enable 1 A	autorise l'appel de l'ISR

BIT DE TCCR1B	7	6	5	4	3	2	1	0
Nom	ICNC1	ICES1	-	WGM13	**WGM12**	**CS12**	**CS11**	**CS10**
Notre valeur	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**1**

CS12 CS11 CS10	000	001	010	011	100	101
Prescaler	arrêté	1	8	**64**	**256**	1024

```
void initTimer1() {
  noInterrupts();                          // on bloque les interruptions pendant le réglage
  TCCR1A = 0;  TCCR1B = 0;                 // on remet les réglages à zéro
  TCNT1  = 0;                              // le compteur commence à 0
  OCR1A  = 1249;                           // 1250 ticks = 5 ms
  TCCR1B |= (1 << WGM12);                  // mode CTC
  TCCR1B |= (1 << CS11) | (1 << CS10);     // prescaler 64, le timer démarre
  TIMSK1 |= (1 << OCIE1A);                 // on autorise l'interruption
  interrupts();
}

ISR(TIMER1_COMPA_vect) {                   // appelée toutes les 5 ms
  digitalWrite(DIG_D2, HIGH);              // 1) on éteint tout
  digitalWrite(DIG_D3, HIGH);
  digitalWrite(DIG_D4, HIGH);
  PORTA = affichage[digitCourant];         // 2) les segments du chiffre
  digitalWrite(pinDigits[digitCourant], LOW);   // 3) on allume cet afficheur
  digitCourant++;                          // 4) au prochain appel, le suivant
  if (digitCourant >= 3) digitCourant = 0;
}
```

ATTENTION

Une ISR doit être courte. Il ne faut jamais mettre de `delay()` ni de `Serial.print()` dedans. Serial utilise lui-même les interruptions, donc le programme peut se bloquer.

5 Le seuil d'alarme et la LED

Q2.4 Le potentiomètre

Un potentiomètre est un diviseur de tension qu'on règle à la main. Le curseur donne une tension entre 0 et la tension du haut. Dans Proteus, le haut est relié à une source de 1,1 V, comme la référence de l'ADC. Donc quand on tourne le curseur d'un bout à l'autre, N va de 0 à 1023.

On veut un seuil entre 0 et 40,0 °C, en dixièmes comme la température, donc entre 0 et 400. La fonction `map()` fait cette règle de trois :

```
int mesureTrigger() {
  return map(analogRead(POT), 0, 1023, 0, 400);   // 512 donne 200, soit 20,0 °C
}
```

> **SUR LA VRAIE CARTE**
> Si on branche le potentiomètre sur 5 V, l'ADC arrive à 1023 dès 1,1 V et seul le premier quart de la course est utile. Pour corriger ça, on ajoute une résistance entre le 5 V et le potentiomètre, d'environ 3,3 fois sa valeur (3,3 kΩ pour un potentiomètre de 1 kΩ).

Q2.5 Le clignotement avec le Timer 3

Le Timer 3 se règle comme le Timer 1, il suffit de remplacer le 1 par 3 dans les noms. Ce qui change, c'est la période et le prescaler.

```
void initTimer3() {
  noInterrupts();
  TCCR3A = 0;  TCCR3B = 0;  TCNT3 = 0;
  OCR3A  = 31249;                          // 31 250 ticks de 16 µs = 0,5 s
  TCCR3B |= (1 << WGM32);                  // mode CTC
  TCCR3B |= (1 << CS32);                   // prescaler 256
  TIMSK3 |= (1 << OCIE3A);
  interrupts();
}

ISR(TIMER3_COMPA_vect) {                   // appelée toutes les 0,5 s
  if (alarme) digitalWrite(LED, !digitalRead(LED));   // on inverse l'état de la LED
  else        digitalWrite(LED, LOW);
}
```

Dans `loop()`, on ne touche plus à la LED. On calcule seulement `alarme = (temp > temp_trigger);`. Si `loop()` écrivait aussi sur la LED, elle entrerait en conflit avec le timer et le clignotement ne serait plus régulier.

6 Le programme complet

C'est la version pour Proteus : afficheur à anode commune avec les 4049, LM35 sur A0 et référence de 1,1 V.

```
/*****************************************************************
 *  TP1 - LM35 & afficheur 7 segments - Arduino Mega 2560 (16 MHz)
 *  Q2.1 mesure | Q2.2 affichage (Timer 1) | Q2.4 seuil | Q2.5 LED (Timer 3)
 *****************************************************************/
#define LM35_pin A0      // capteur LM35
#define POT      A1      // potentiomètre du seuil
#define LED      32      // LED d'alarme
#define DIG_D2   52      // afficheur des dizaines
#define DIG_D3   51      // afficheur des unités (+ point)
#define DIG_D4   50      // afficheur des dixièmes

// Codes 0..9, 1 = segment allumé (vu de l'Arduino, grâce au 4049)
byte tabDigit[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
const byte pinDigits[3] = {DIG_D2, DIG_D3, DIG_D4};

volatile byte affichage[3];          // les 3 codes à afficher
volatile byte digitCourant = 0;      // afficheur allumé en ce moment
volatile bool alarme = false;        // décidée par loop(), lue par Timer 3

int temp;                            // température en dixièmes (248 = 24,8 °C)
int temp_trigger;                    // seuil en dixièmes (0..400)

void prepareAffichage(int number);
int  mesureTemp();
int  mesureTrigger();
void initTimer1();
void initTimer3();

void setup() {
  Serial.begin(9600);

  pinMode(DIG_D2, OUTPUT);  digitalWrite(DIG_D2, HIGH);   // HIGH = éteint
  pinMode(DIG_D3, OUTPUT);  digitalWrite(DIG_D3, HIGH);
  pinMode(DIG_D4, OUTPUT);  digitalWrite(DIG_D4, HIGH);
  pinMode(LED, OUTPUT);     digitalWrite(LED, LOW);

  analogReference(INTERNAL1V1);      // 1 pas = 1,07 mV ≈ 0,1 °C
  analogRead(LM35_pin);              // 1re lecture ignorée

  DDRA  = 0xFF;                      // broches 22..29 en sortie
  PORTA = 0x00;

  initTimer1();                      // multiplexage toutes les 5 ms
  initTimer3();                      // clignotement toutes les 0,5 s

  prepareAffichage(888);             // test des segments : 88.8
  delay(3000);
}

void loop() {
  temp         = mesureTemp();
  temp_trigger = mesureTrigger();
  alarme       = (temp > temp_trigger);   // loop() décide, les timers agissent
```

```
  prepareAffichage(temp);

  Serial.print("Temp = ");
  Serial.print(temp / 10);  Serial.print(".");  Serial.print(temp % 10);
  Serial.print(" C  |  Seuil = ");
  Serial.print(temp_trigger / 10);  Serial.print(".");  Serial.print(temp_trigger % 10);
  Serial.println(alarme ? " C  -> ALARME" : " C");

  delay(500);
}

// Température en dixièmes : N x 1100 / 1024 = mV = dixièmes de °C
int mesureTemp() {
  long N = analogRead(LM35_pin);          // long : 1023 x 1100 dépasse un int
  return (int)((N * 1100L) / 1024L);
}

// Seuil en dixièmes : 0..1023 -> 0..400 (0,0 à 40,0 °C)
int mesureTrigger() {
  return map(analogRead(POT), 0, 1023, 0, 400);
}

// Découpe la valeur en 3 chiffres et prépare les codes
void prepareAffichage(int number) {
  if (number > 999) number = 999;         // 3 digits : 99,9 max
  if (number < 0)   number = 0;

  int dizaine = number / 100;
  int unite   = (number / 10) % 10;
  int dixieme = number % 10;

  if (dizaine == 0) affichage[0] = 0x00;   // pas de zéro inutile
  else              affichage[0] = tabDigit[dizaine];
  affichage[1] = tabDigit[unite] | 0x80;   // point décimal (bit 7)
  affichage[2] = tabDigit[dixieme];
}

// Timer 1 : CTC, prescaler 64, 1250 ticks de 4 us = 5 ms
void initTimer1() {
  noInterrupts();
  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1  = 0;
  OCR1A  = 1249;
  TCCR1B |= (1 << WGM12);                 // mode CTC
  TCCR1B |= (1 << CS11) | (1 << CS10);    // prescaler 64
  TIMSK1 |= (1 << OCIE1A);                // autorise l'interruption
  interrupts();
}

// Timer 3 : CTC, prescaler 256, 31250 ticks de 16 us = 0,5 s
void initTimer3() {
  noInterrupts();
  TCCR3A = 0;
  TCCR3B = 0;
  TCNT3  = 0;
  OCR3A  = 31249;
  TCCR3B |= (1 << WGM32);                 // mode CTC
  TCCR3B |= (1 << CS32);                  // prescaler 256
  TIMSK3 |= (1 << OCIE3A);
  interrupts();
}
```

```
// Multiplexage : un afficheur à chaque appel (toutes les 5 ms)
ISR(TIMER1_COMPA_vect) {
  digitalWrite(DIG_D2, HIGH);              // 1) tout éteindre
  digitalWrite(DIG_D3, HIGH);
  digitalWrite(DIG_D4, HIGH);

  PORTA = affichage[digitCourant];         // 2) segments du chiffre
  digitalWrite(pinDigits[digitCourant], LOW);   // 3) allumer cet afficheur

  digitCourant++;                          // 4) le suivant au prochain appel
  if (digitCourant >= 3) digitCourant = 0;
}

// Clignotement : bascule la LED toutes les 0,5 s si alarme
ISR(TIMER3_COMPA_vect) {
  if (alarme) digitalWrite(LED, !digitalRead(LED));
  else        digitalWrite(LED, LOW);
}
```

Q2.3 Sur la vraie carte

- Avec un afficheur à cathode commune (par exemple 5641AS) branché directement, le code marche sans modification.
- Avec un afficheur à anode commune (par exemple 5641BS) branché directement, il faut inverser les deux logiques : `PORTA = ~affichage[…]` , le digit sélectionné passe à `HIGH` et pour tout éteindre on met `LOW` .
- On met une résistance d'environ 1 kΩ sur chaque segment (broches 22 à 29), et rien sur les digits. Une broche de l'Arduino ne supporte pas plus de 40 mA.
- Pour le LM35, face plate vers soi : à gauche +5 V, au milieu Vout, à droite GND. S'il chauffe, c'est qu'il est branché à l'envers.
- On teste étape par étape : d'abord la mesure seule avec le moniteur série à 9600 bauds, ensuite le test 88.8, et à la fin le potentiomètre et la LED.

7 Les erreurs qu'on rencontre souvent

CE QU'ON OBSERVE	LA CAUSE	LA CORRECTION
Température bizarre ou négative	le calcul dépasse un `int`	`long N` et `1100L`
Le moniteur série défile trop vite	pas de `delay()` dans `loop()`	ajouter `delay(500)`
Les chiffres apparaissent à l'envers (en négatif)	confusion entre anode et cathode commune	`~` sur le code, ou l'autre table
Les 3 afficheurs montrent le même chiffre	plusieurs digits à LOW en même temps	tout éteindre avant d'allumer le suivant
On voit un reflet du mauvais chiffre	PORTA changé avant d'éteindre	éteindre, écrire PORTA, puis allumer
Une seule barre verticale au lieu du chiffre	on a envoyé le chiffre et pas son code	`tabDigit[chiffre]`
Une barre en bas au lieu du point	`0x08` au lieu de `0x80`	`| 0x80`
Affichage éteint, l'ISR n'est jamais appelée	la ligne `TIMSKn |= (1 << OCIEnA)` manque	l'ajouter
La LED reste faiblement allumée sans clignoter	on a gardé la période de 5 ms du Timer 1	prescaler 256 et `OCR3A = 31249`
Le programme se bloque	`Serial.print()` dans une ISR	l'afficher dans `loop()`
Erreur « not declared in this scope »	la fonction est écrite après l'endroit où on l'appelle	la déclarer en haut du fichier
Erreur `expected ';'`	un `:` à la place d'un `;`	regarder la ligne d'avant

8 Exercices

1. Le LM35 sort 0,375 V. Quelle est la température ? Quel nombre N donne l'ADC avec la référence 1,1 V ?
2. Avec la référence 5 V, de combien de degrés la mesure change quand N augmente de 1 ?
3. Pour `t = 375`, que valent `t / 100`, `(t / 10) % 10` et `t % 10` ?
4. Calcule le code du chiffre 6, puis celui du « 7. » avec le point.
5. Qu'est-ce qu'on voit sur l'afficheur si on écrit `tabDigit[4] | 0x08` ?
6. On part de `x = 0000 0000`. Donne la valeur de x après `x |= (1 << 3);`, puis après `x |= (1 << 1) | (1 << 0);`, puis après `x &= ~(1 << 3);`.
7. Quel prescaler et quelle valeur d'OCR pour avoir une interruption toutes les 1 s ? Et toutes les 100 ms ?
8. Avec `OCR1A = 249` et un prescaler de 64, quelle est la période ?
9. Que se passe-t-il si on oublie la ligne avec `OCIE1A` ? Et si on oublie celle avec `CS11 | CS10` ?
10. Pourquoi l'affichage reste allumé pendant le `delay(500)` de `loop()` ?

Corrigé

1. 375 / 10 = 37,5 °C. Et N = 375 × 1024 / 1100 ≈ 349.
2. 4,88 / 10 ≈ 0,49 °C. C'est trop pour afficher les dixièmes.
3. 3, 7 et 5, c'est-à-dire 37,5 °C.
4. Le 6 c'est A C D E F G, donc 0x7D. Le « 7. » donne 0x07 | 0x80 = 0x87.
5. 0x66 | 0x08 = 0x6E. On voit un 4 avec une barre en bas (segment D), et pas de point.
6. 0000 1000, puis 0000 1011, puis 0000 0011.
7. Pour 1 s : prescaler 256, 62 500 ticks, OCR = 62 499. Pour 100 ms : prescaler 64, 25 000 ticks, OCR = 24 999.
8. (249 + 1) × 64 / 16 000 000 = 1 ms.
9. Sans OCIE1A, le timer compte mais n'appelle jamais l'ISR, donc l'affichage reste éteint. Sans CS11 | CS10, le timer ne démarre même pas.
10. Parce que l'interruption du Timer 1 coupe le `delay()` toutes les 5 ms pour passer à l'afficheur suivant.

★ Résumé du TP

1. La mesure

- LM35 : 10 mV par °C, alimenté en 5 V.
- ADC sur 10 bits : N va de 0 à 1023.
- Référence 1,1 V : 1 pas = 1,07 mV, environ 0,1 °C.
- N × 1100L / 1024 donne des mV, donc des dixièmes de °C.
- On découpe avec `/` et `%`.

2. Le codage d'un chiffre

- Un bit par segment, du bit 0 (A) au bit 7 (DP).
- Les segments sont sur le PORTA (broches 22 à 29).
- Le point : `| 0x80`.
- Dans Proteus (anode commune + 4049) : segment à HIGH = allumé, digit à LOW = sélectionné.

3. Le multiplexage

- Les segments sont partagés, donc un seul afficheur à la fois.
- On éteint, on écrit PORTA, puis on allume.
- 5 ms par afficheur, 67 Hz, l'œil voit tout allumé.

4. Les timers

- Compteur matériel en mode CTC, avec une ISR à chaque fois qu'il atteint la cible.
- Temps = nombre de ticks × (P / 16) µs.
- OCR = (16 000 000 / P) × temps − 1, sans dépasser 65 535.
- Timer 1 : P = 64 et OCR1A = 1249 pour 5 ms.
- Timer 3 : P = 256 et OCR3A = 31249 pour 0,5 s.
- On ne touche pas au Timer 0.

5. Le seuil et la LED

- `map(N, 0, 1023, 0, 400)` pour 0 à 40,0 °C.
- `alarme = temp > seuil;`
- Le Timer 3 fait clignoter la LED.

6. Quelques règles

- `volatile` pour les variables utilisées dans une ISR.
- Pas de `delay` ni de `Serial` dans une ISR.
- `|=` et `&= ~` pour modifier un registre.

> **À RETENIR**
> `loop()` mesure et décide, les timers s'occupent du reste. Le Timer 1 gère l'affichage et le Timer 3 fait clignoter la LED.

RHIBA Zakaria
Zakpy Academy, formation Arduino et systèmes embarqués. Bon courage !